COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/14/2026	**DATE SUBMITTED:**	10/14/2026
NAME:	Katindig, Jiro Luis Demain, Jaime Luis	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12.1 **Algorithmic Strategies**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

Strategy	Algorithm	Analysis
Recursion	Tree Traversals	Function calls itself on small subtrees (left->inOrder(), right->inOrder()) until the base case (nullptr) is encountered, satisfying the requirement of self-similar decomposition.
Brute Force	Element Uniqueness Check	Uses nested loops to examine each possible pair of elements exhaustively, without any heuristic pruning or optimization through sorting beforehand.
Backtracking	Depth-First Search	Finds its way down a single path through adjacent vertices to the point where there is no way forward or the vertex has already been visited, and then goes back up to the last visited vertex with an unvisited neighbor.
Greedy	Selection Sort	In each iteration, it finds the largest element among the unsorted array by scanning and makes its decision by swapping elements permanently.
Divide-and-Conquer	Binary Search / Merge Sort	Divides the problem into two equal parts (mid = low + (high - low) / 2) and discards the part not relevant to solving the problem, solving the problem recursively or iteratively without overlapping subproblems.

Table 12-1. Algorithmic Strategies and Examples

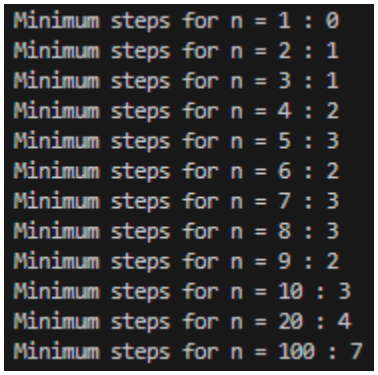
Screenshot	
Analysis	The top-down approach involves recursively decomposing (n) starting from the root problem. As the calculated state of each subproblem from 1 to (n) is stored in the memo table with initial value set to -1, no subproblem will be calculated more than once. The recursive decomposition from exponential O(3^n) to linear O(n) comes at the cost of overhead caused by program execution call stack when very large (n) is used, as it may cause stack overflow eventually.

Table 12-2. Memoization Implementation

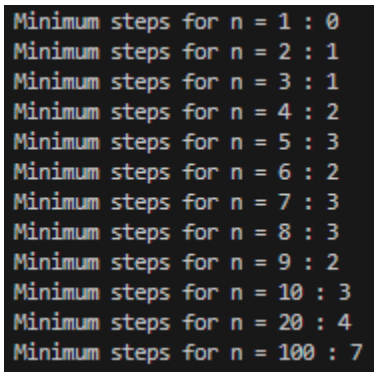
Screenshot	
Analysis	The bottom-up approach involves building solutions iteratively from the base problem (dp[1] = 0) to the target (n) using a linear loop. At each index (i), it finds out the minimum cost of transition using previously determined values (dp[i - 1], dp[i / 2], dp[i / 3]). In terms of output, space and time complexity, this approach matches the previous one with O(n) but without any function call recursion overhead and stack overflow limit at all.

Table 12-3. Bottom-Up Dynamic Programming Implementation

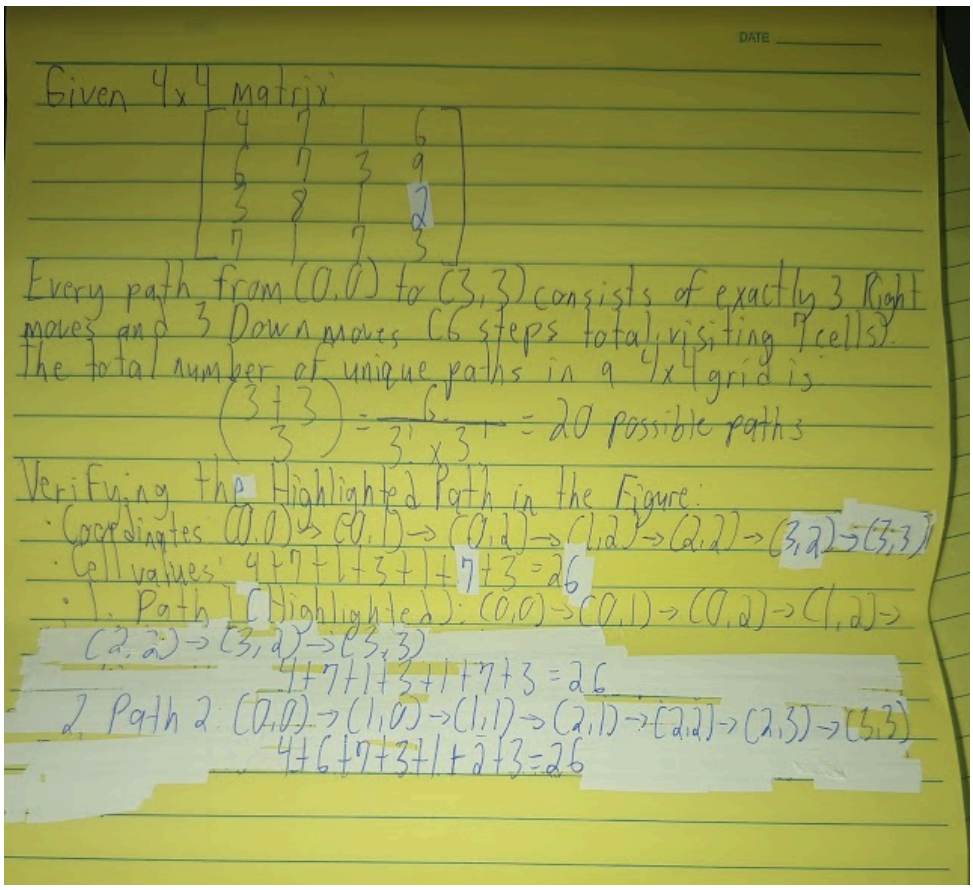
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Pseudocode	Algorithm countPaths(mat, m, n, cost):
	// Base Case 1: Out of bounds or cost exceeded
	if m < 0 or n < 0 or cost < 0:
	return 0
	
	// Base Case 2: Reached the origin (0, 0)
	if m == 0 and n == 0:
	if mat[0][0] == cost:
	return 1
	else:
	return 0
	
	// Check memoization table if already computed
	if memo[m][n][cost] is present:
	return memo[m][n][cost]
	
	// Recursive Step: Arrive at (m, n) from top (m-1, n) or left (m, n-1)
	pathsFromTop = countPaths(mat, m - 1, n, cost - mat[m][n])
	pathsFromLeft = countPaths(mat, m, n - 1, cost - mat[m][n])
	
	memo[m][n][cost] = pathsFromTop + pathsFromLeft
	return memo[m][n][cost]

Solving Problem



C++ Code

```
#include <iostream>
#include <vector>
#include <string>
#include <unordered_map>

#define M 4
#define N 4

// Dynamic Programming with Memoization using an
unordered_map key
int countPaths(int mat[M][N], int m, int n, int cost,
std::unordered_map<std::string, int>& dp) {
    if (m < 0 || n < 0 || cost < 0) {
```

```
        return 0;
    }

    if (m == 0 && n == 0) {
        return (mat[0][0] == cost) ? 1 : 0;
    }

    std::string key = std::to_string(m) + "|" +
std::to_string(n) + "|" + std::to_string(cost);

    if (dp.find(key) != dp.end()) {
        return dp[key];
    }


    int pathsFromTop = countPaths(mat, m - 1, n, cost -
mat[m][n], dp);
    int pathsFromLeft = countPaths(mat, m, n - 1, cost -
mat[m][n], dp);

    dp[key] = pathsFromTop + pathsFromLeft;
    return dp[key];
}

int main() {
    int mat[M][N] = {
        { 4, 7, 1, 6 },
        { 6, 7, 3, 9 },
        { 3, 8, 1, 2 },
        { 7, 1, 7, 3 }
    };

    std::cout << "--- Count Paths in Matrix with Given
Cost ---" << std::endl;
    std::cout << "Matrix Size: " << M << " x " << N <<
std::endl;
```

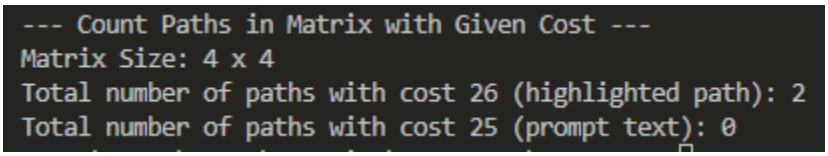
```
    // Test with cost = 26
    std::unordered_map<std::string, int> dp26;
    int target_cost1 = 26;
    int result1 = countPaths(mat, M - 1, N - 1,
target_cost1, dp26);
    std::cout << "Total number of paths with cost " <<
target_cost1
              << " (highlighted path): " << result1 <<
std::endl;

    // Test with cost = 25
    std::unordered_map<std::string, int> dp25;
    int target_cost2 = 25;
    int result2 = countPaths(mat, M - 1, N - 1,
target_cost2, dp25);
    std::cout << "Total number of paths with cost " <<
target_cost2
              << " (prompt text): " << result2 <<
std::endl;

    return 0;
}
```

Analysis	**Katindig:** This program counts how many grid paths match a target cost by starting at the end cell and working backward to the start. As it moves, it subtracts each cell's number from the total cost. To stay fast, it saves every result it calculates into a lookup table. If it reaches the same cell with the same remaining cost again, it reuses the saved result instead of doing the math all over again. This avoids repeated work and makes the code run much faster. **Demain:** This approach breaks the problem down by finding paths to each cell from its top or left neighbor, subtracting the current cell's value from the remaining cost. A basic recursive search would be very slow, taking exponential O(2^(M+N)) time because it re-checks the same paths over and over. By saving each solved state in a lookup table, the program

	calculates each unique cell and cost combination only once. This lowers the total time complexity to O(M * N * Cost), using O(M * N * Cost) space to store saved results plus O(M + N) space for the call stack.

Screenshot	

Conclusion	**Katindig:** In this hands-on activity, I've gained a much clearer understanding of how different algorithmic strategies work and when to apply them into solving programming problems. I've recalled that simple strategies like basic recursion and brute force are easy to write, but they can quickly become slow and inefficient when they repeat the exact same calculations over and over as I compared it to others. Through our hands-on tasks with the problems in the procedure (ILO A), I saw how Dynamic Programming fixes this problem. By storing our calculated results in a memo lookup table or a bottom-up array, we ensure that no subproblem is ever calculated twice. This greatly helps cut down execution time and prevents memory stack overflows on larger inputs. Overall, this lab taught me that writing good code is not just about getting the right answer, but also about picking the right algorithmic strategy to make programs run as efficiently as possible. **Demain:** This HOA was essential in providing practical insight on how to identify, implement, and compare different modern algorithmic solutions, particularly those that require the usage of dynamic programming due to the problems that involve overlapping subproblems and optimal substructure. In the process of the lab, finding the "Minimum Steps to One" was successful in revealing that a greedy approach does not lead to an optimal solution to the problem, but top-down dynamic programming and bottom-up dynamic programming reduce the exponential O(3n) tree to linear O(n) by memorizing intermediate subproblems. The extra exercise on the problem of counting paths in matrices was successful in showing that a 2D grid can be

	explored efficiently without having any branching issues by formulating recurrence relationships based on previous adjacent cells and storing state keys. On the whole, I demonstrated good proficiency in formulating recurrence relations, creating memoized structures in C++ code, and understanding differences between recursion-based and table construction approaches. However, the major area where I need to improve my skills is related to generating bottom-up multidimensional dynamic programming tables directly from recursive models without using memoization first.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **For assistance in parts of the procedure and forming a supplementary activity to understand its contents and improve grammar in forming an analysis and conclusion.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI firstly to understand the concepts of the procedure part more and also improve the grammar of the analysis of it. Second, to help me understand the supplementary activity. It's because I need to have more clarity on what each ILO of the supplementary is to answer it precisely. Lastly, I used AI to improve my grammar and tone in forming my conclusion.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.